

--carefully-skip-permissions

Безопасный Auto Mode для Kilo Code

Проект переносит функциональность Claude Code Auto Mode в open-source-агент Kilo Code. Система перехватывает действия агента до выполнения и автоматически возвращает *allow*, *deny* или *escalate* — без перехода к небезопасному blanket auto-approval.

ядро Auto Mode — реализуется

открытый benchmark в sandbox — в работе

КРАТКАЯ АННОТАЦИЯ

Что делает проект

Проект переносит функциональность Claude Code Auto Mode в open-source-агент Kilo Code. Система перехватывает действия агента до выполнения и автоматически возвращает **allow**, **deny** или **escalate**, чтобы снизить число ручных подтверждений без перехода к небезопасному blanket auto-approval.

Сейчас команда работает над двумя задачами: завершает интеграцию Auto Mode в Kilo Code и настраивает воспроизводимый запуск агента в открытом benchmark-харнессе.

СЕЙЧАС В РАБОТЕ

интеграция Auto Mode в Kilo Code

воспроизводимый benchmark-харнесс

slopsquatting — исключён из scope

ПРОБЛЕМАТИКА

Approval fatigue против blanket auto-approval

Пользователь coding agent выбирает между двумя неудобными режимами.

Подтверждать почти каждое действие

Создаёт approval fatigue: пользователь устаёт кликать «разрешить» и теряет внимательность именно там, где она нужна.

Полностью отключить проверки

Позволяет агенту выполнять неосторожные действия или действия, вызванные prompt injection, без какого-либо контроля.

Для open-source-агентов проблема особенно актуальна: в корпоративных контурах часто используются локальные модели, а развитого аналога Claude Auto Mode в Kilo Code пока нет.

ОСНОВНЫЕ РАССМАТРИВАЕМЫЕ РИСКИ

- деструктивные команды
- действия вне намерения пользователя
- indirect prompt injection
- эксфильтрация данных
- обход проверки сложной командой

АКТУАЛЬНАЯ ПОСТАНОВКА ЗАДАЧИ

Что нужно сделать

- 1 Перехватывать запрос агента до исполнения.
- 2 Оценивать действие с учётом пользовательского запроса и контекста сессии.
- 3 Автоматически разрешать безопасное, блокировать опасное и передавать спорное человеку.
- 4 Возвращать агенту структурированный отказ, чтобы он мог продолжить задачу безопасным способом.
- 5 Сравнить baseline и защищённый режим на одном открытом benchmark в одинаковом sandbox.

Модель угроз

Предполагается добросовестный пользователь.
Недоверенными считаются файлы проекта, результаты команд, MCP responses и веб-контент.

Защита от изначально злонамеренного запроса пользователя не входит в основной score.

ВЫБРАННОЕ ТЕХНИЧЕСКОЕ РЕШЕНИЕ

Permission adjudicator

- 1

tool call
агент запрашивает действие
- 2

Permission.ask
перехват до выполнения
- 3

host-built facts
факты о действии формирует Kilo Code, а не модель
- 4

правила и circuit breakers
жёсткие правила имеют приоритет над решением классификатора
- 5

LLM-классификатор
только для серых случаев; ограниченный контекст, строгая схема ответа
- 6

trusted resolver
формирует итоговое решение
- 7

вердикт

allow

deny

escalate

в unattended-сессии escalate становится отказом, а не машинным разрешением
- 8

аудит
решения журналируются для расчёта ASR, Utility, Friction и Latency

ВЫБРАННОЕ ТЕХНИЧЕСКОЕ РЕШЕНИЕ

Ключевые свойства

- Факты о действии формирует Kilo Code, а не модель.
- Классификатор не видит chain-of-thought, обычные ответы агента и содержимое tool outputs — только сообщения пользователя, host-built facts текущего вызова и компактную историю без вывода.
- Жёсткие правила имеют приоритет над решением классификатора.
- Классификатор получает ограниченный контекст и отвечает по строгой схеме.
- В unattended-сессии escalate становится отказом, а не машинным разрешением.
- Решения журналируются для расчёта ASR, Utility, Friction и Latency.

Два независимых флага

--adjudicate — защищённый режим адьюдикации.

--auto — существующий режим, автоматически одобряющий разрешения.

Детали зафиксированы в техническом дизайн-документе

2026-09-03-kilo-auto-mode-design.md

ВЫБРАННОЕ ТЕХНИЧЕСКОЕ РЕШЕНИЕ

Почему классификатор «не видит» рассуждения

Reasoning-blind контекст уменьшает поверхность prompt injection: вредоносная инструкция из README, MCP response или вывода команды не передаётся классификатору напрямую и не может «уговорить» его через рассуждения основного агента.

Исключение. Короткая ссылка на непосредственно предшествующий ответ ассистента нужна для интерпретации ответов пользователя вроде «да». Она считается недоверенным контекстом и *сама по себе не авторизует действие*.

Это повышает устойчивость архитектуры, но не является абсолютной гарантией: инъекция всё ещё может повлиять на аргументы tool call, поэтому окончательное решение ограничивают правила и trusted resolver.

УЖЕ РЕАЛИЗОВАННЫЕ КОМПОНЕНТЫ

Что готово в рабочей ветке

- Adjudicator и trusted resolver с вердиктами allow, deny, escalate.
- Каталог из 50 hard, soft и exempt правил.
- Circuit breakers для ряда деструктивных и непрозрачных действий.
- Сбор фактов для shell, файловых операций, MCP и других каналов Kilo Code.
- Строгая валидация ответа LLM-классификатора.
- Интеграция в Permission.ask, CLI и TUI.
- Structured refusal и продолжение после блокировки.
- Хранение политик и журнала решений.
- Набор модульных и интеграционных тестов.

Benchmark-харнесс пока не готов: команда настраивает adapter запуска Kilo Code и sandbox.

ПРЕДВАРИТЕЛЬНЫЕ РЕЗУЛЬТАТЫ

Что показывает текущее состояние ветки

100 / 100

bun test test/kilocode/automode — тестов
пройдено

0

ошибок в тестовом прогоне

OK

bun run typecheck для packages/opencode

Тестами покрыты приоритет hard-правил, защита host-built facts, structured output классификатора и передача отказа агенту. Отдельным тестом подтверждено, что prompt классификатора не содержит tool outputs и внутренних рассуждений агента.

Это подтверждает техническую целостность прототипа, но не его безопасность. Значения ASR, Utility, FP/FN, Friction и Latency появятся после внешнего benchmark-прогона.

H1 и H2

H1

Auto Mode снижает permission friction

Если безопасные действия разрешать автоматически, опасные блокировать, а человеку показывать только спорные, число ручных подтверждений снизится без нежелательного роста ASR.

ПРОВЕРКА — сравнить `--auto` и `--adjudicate` по ASR, Utility, Friction, FP/FN и Latency на одинаковых задачах.

H2

Kilo Code можно воспроизводимо оценивать открытым benchmark

Если зафиксировать версию агента, модель, состояние sandbox, параметры запуска и критерии фактического ущерба, результаты нескольких прогонов будут сопоставимы.

ПРОВЕРКА — несколько раз запустить baseline, измерить разброс и проверить файловые и сетевые сапай-индикаторы.

ПРОДУКТОВЫЕ И ИССЛЕДОВАТЕЛЬСКИЕ ГИПОТЕЗЫ

Н3 — нужны разные уровни Security Effort

Профили Low, Medium и High позволят адаптировать цену и строгость проверки к риску проекта лучше, чем единая политика.

ПРОФИЛЬ	КОНТЕКСТ	ПРЕДПОЛАГАЕМОЕ ПОВЕДЕНИЕ
Low	Локальный проект без credentials	Минимум проверок и model calls, базовые circuit breakers.
Medium	Проект с credentials или приватными данными	Больше контекста, строгая проверка внешних действий.
High	Production и критичные данные	Максимальный security effort и консервативные пороги.

Гипотеза проверяется по безопасности, Utility, Friction, Latency и стоимости. High нельзя заранее считать лучшим: чрезмерная строгость может сделать режим непригодным.

ОГРАНИЧЕНИЯ

Что пока не решено

- Внешний benchmark ещё не запущен, поэтому количественных security-результатов нет.
- LLM-классификатор остаётся вероятностным и зависит от выбранной модели.
- Детерминированные правила не покрывают все варианты shell и внешних API.
- Анализ отдельного действия может пропустить опасную последовательность действий.
- Fail-closed на неизвестных случаях повышает Friction.
- Security Effort пока существует только как продуктовая гипотеза.
- Slopsquatting, cloud control planes и Kubernetes не входят в текущую версию.
- Более безопасный Auto Mode может повысить доверие пользователей и общий объём автономных действий — снижение ASR не гарантирует снижения суммарного риска.

ОТКРЫТЫЕ ВОПРОСЫ

Что предстоит решить команде

- Какой открытый benchmark и какую его версию использовать?
- Как стабильно запускать Kilo Code без интерактивного UI?
- Как настроить sandbox и достоверно фиксировать canary-эффекты?
- Какой контекст нужен классификатору: один action или часть траектории?
- Какие проверки и модели должны соответствовать профилям Low, Medium, High?

ПЛАН РАБОТ ДО ФИНАЛЬНОЙ ВЕРСИИ

Пять шагов до готового результата

- 1 Стабилизировать Auto Mode**
Завершить интеграцию, проверить attended/unattended сценарии и подготовить demo allow / deny / escalate.
- 2 Запустить benchmark**
Реализовать adapter Kilo Code, настроить очищаемый sandbox и получить baseline.
- 3 Провести сравнение**
Измерить ASR, Utility, FP/FN, Friction и Latency, разобрать ошибки и выполнить ablation правил и классификатора.
- 4 Проверить Security Effort**
Описать минимальные профили и сравнить их хотя бы на репрезентативном подмножестве задач.
- 5 Упаковать результат**
Зафиксировать версии кода, модели и benchmark, подготовить инструкции, таблицу результатов, ограничения и демонстрацию.

КОМАНДА И РАСПРЕДЕЛЕНИЕ ЗАДАЧ

Кто чем занимается

УЧАСТНИК	РОЛЬ	ЗОНА ОТВЕТСТВЕННОСТИ	УЧАСТИЕ
Дыдалин Дмитрий	Разработчик	Основной функционал Auto Mode, интеграция adjudicator, правила и тесты.	активное · Auto Mode
Кеменов Александр	Разработчик	Запуск Kilo Code в открытом benchmark и настройка sandbox.	активное · benchmark
Поляков Владислав	Product	Постановка задачи, продуктовые гипотезы, метрики и упаковка результатов.	активное · продукт